

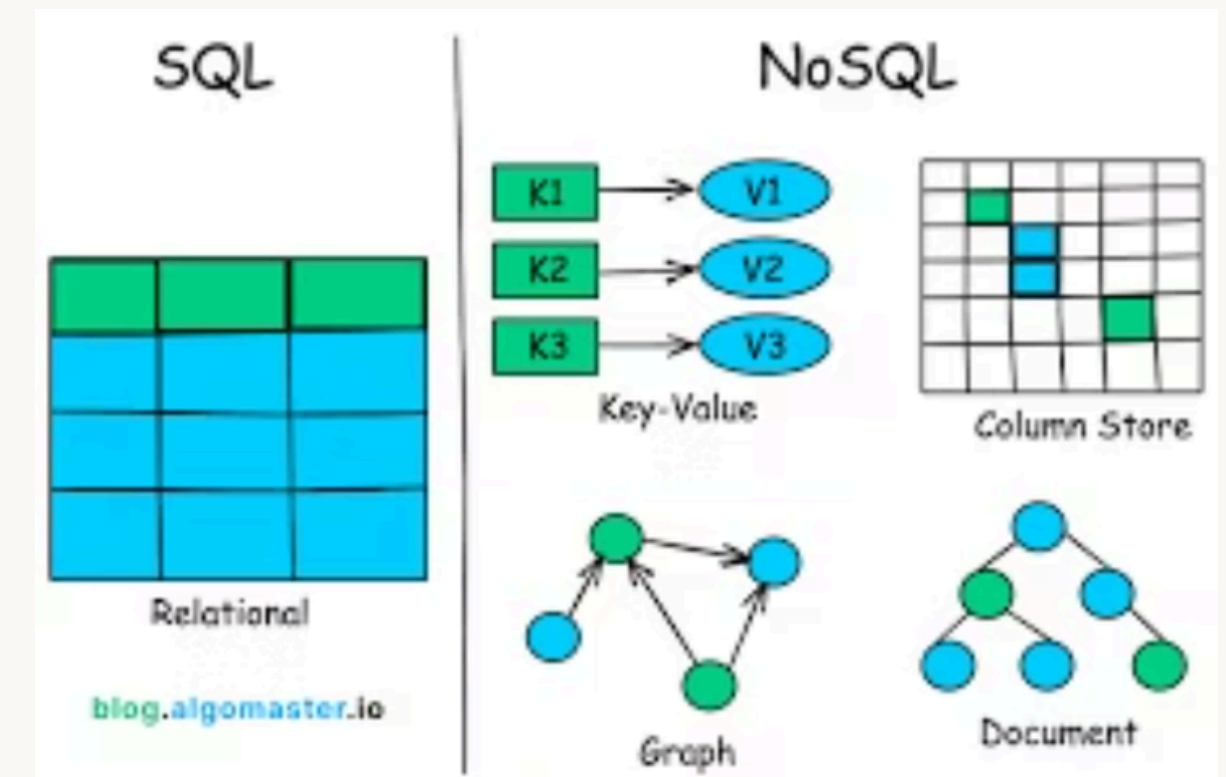
The Emergence of NoSQL

PRESENTATION

Tushar Thakryan (013 1905 1723)
Ronak Dutta (014 1905 1723)

What is NoSQL? ("*Not Only SQL*")

NoSQL, which stands for "**not only SQL**" or "**non-relational**", refers to a category of databases that store and manage data in a format other than **the traditional tabular relations of relational databases**.



Traditional databases couldn't handle the massive amounts of new data the internet created. NoSQL wasn't built to *replace* SQL, it was built to solve **different problems**.

Huge Volume

Handles massive amounts of data with ease

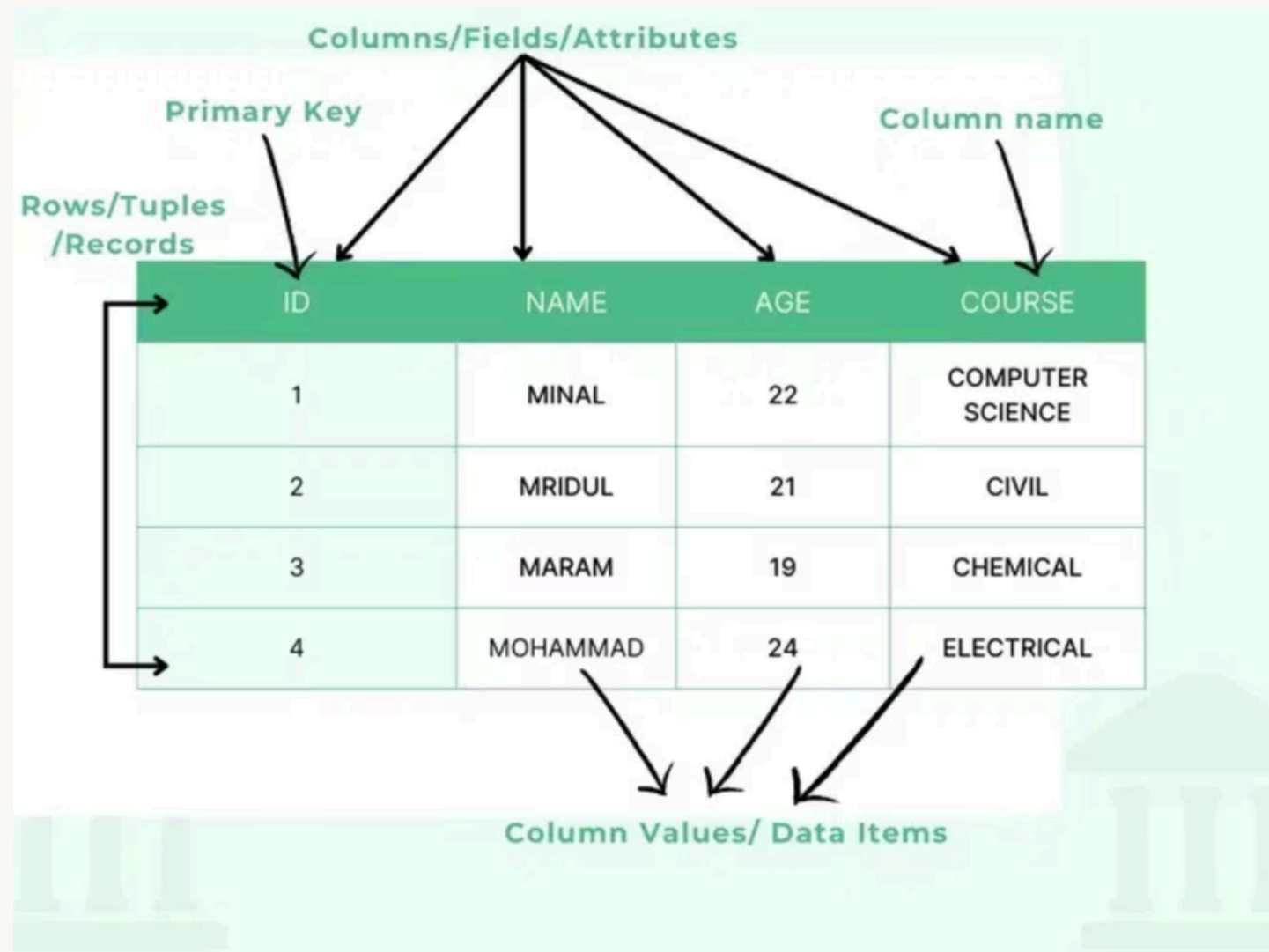
Lightning Fast

Super fast reads and writes at scale

Messy Data

Stores unstructured data that doesn't fit neat tables

Why Traditional RDBMS Struggled



Relational databases work great for banking — but they buckle under the scale of something like Facebook or Amazon. They're simply **too rigid** for the modern web.

Rigid Structure

Fixed schemas make schema changes slow, painful, and risky

Expensive Scaling

Scaling "up" means buying one bigger, costlier machine

Poor Fit for Modern Data

Struggles with images, social media feeds, and unstructured content

Why Data Suddenly Changed

Around the mid-2000s, smartphones went mainstream, social media exploded, and IoT devices appeared everywhere. The result? "**Big Data**" and it changed everything.



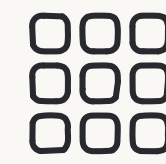
Volume

The sheer **amount** of data grew at an almost unbelievable rate — terabytes became petabytes overnight



Velocity

Data started arriving in **real time** tweets, sensor pings, and clicks faster than old systems could handle



Variety

Data was no longer just text in rows — it was **videos, likes, GPS coordinates,** and sensor readings

The Birth of NoSQL

Around **2005–2010**, Google, Amazon, and Facebook hit a wall — their old databases simply couldn't keep up. So they built their own solutions from scratch, out of pure necessity.

1

Handle Big Data

Built specifically for massive, high-velocity data loads

2

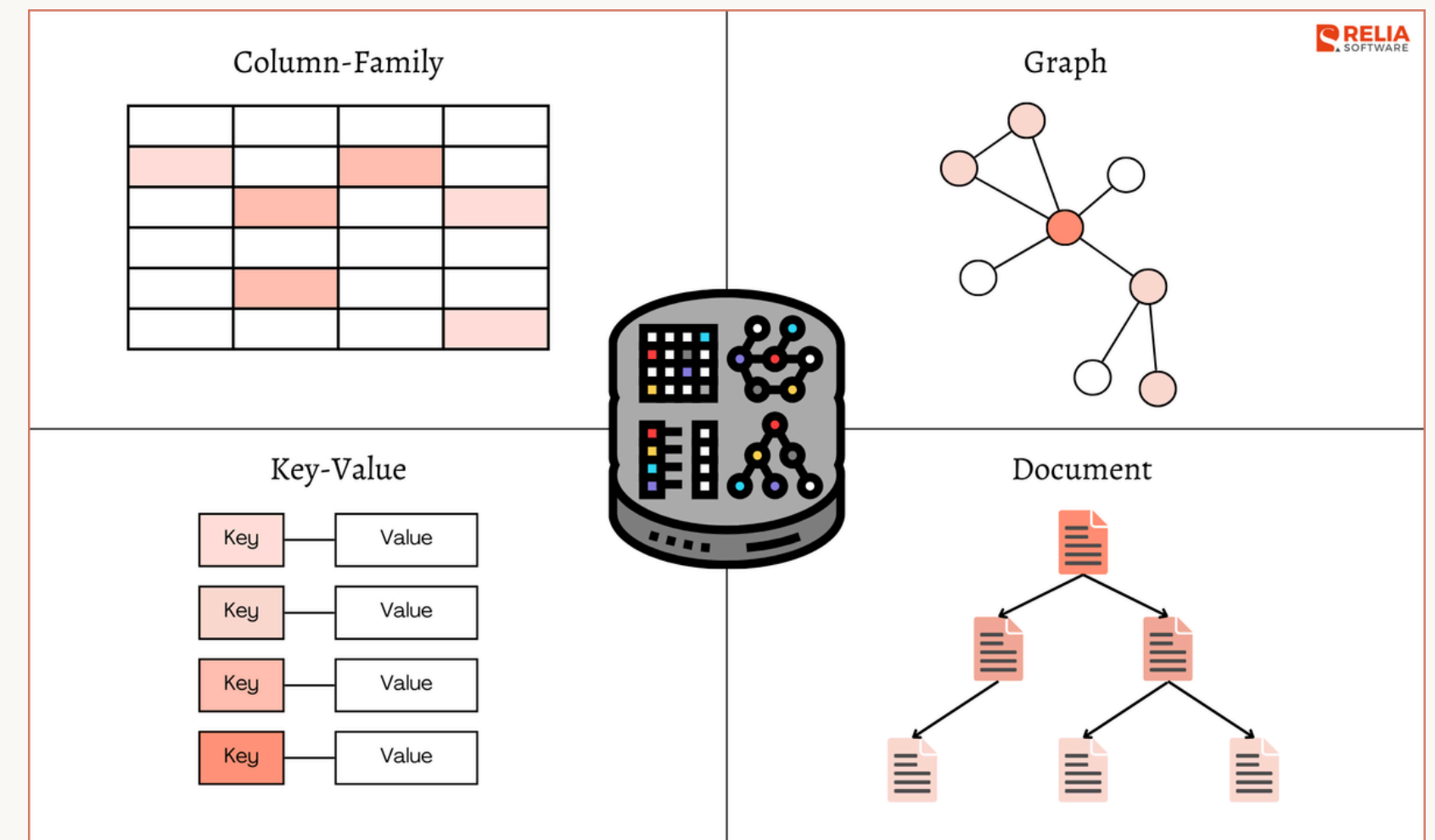
Scale Out

Designed to spread across hundreds of cheap commodity servers

3

Stay Online

Apps keep running even when individual servers fail



Cool Features of NoSQL

NoSQL took off because it gave developers the **flexibility and speed** that modern web apps demand. Here's what makes it stand out.



Flexible Schema

No need to define a rigid structure upfront — just store your data

Horizontal Scaling

Need more power? Just **add more machines** — no downtime required

High Performance

Blazing-fast reads and writes for simple, high-frequency queries

The Four Main Types of NoSQL

NoSQL isn't just one technology — it's a **family of solutions**, each designed for a different kind of problem.



Key-Value Stores

Redis — blazing-fast caching and session management using simple key-value pairs



Document Databases

MongoDB — stores flexible JSON-like documents, perfect for varied data structures



Column-Family Stores

Cassandra — built for planet-scale writes and reads across distributed clusters



Graph Databases

Neo4j — maps relationships between data points, great for social networks

Simple Comparison: Old vs. New

Here's a side-by-side look at how traditional SQL and modern NoSQL differ on the things that matter most.

RDBMS — The Old Way	NoSQL — The New Way
Fixed, rigid schema — structure defined upfront	Flexible schema — store data in any shape
Scales <i>up</i> — buy a bigger, costlier machine	Scales <i>out</i> — add more commodity machines
Structured data only — rows and columns	Any type of data — text, JSON, video, graphs
Strong consistency (ACID transactions)	Tunable consistency — trade-off for availability
Best for: banking, finance, ERP systems	Best for: social media, real-time apps, IoT

 Both approaches are valid, the right choice depends entirely on your use case.

Where is NoSQL Used Today?

You interact with NoSQL **every single day** — it quietly powers the apps you rely on most.



Facebook

Manages **billions** of user profiles, posts, and likes in real time using Cassandra



Amazon

Powers a **lightning-fast** product catalog and shopping cart with DynamoDB



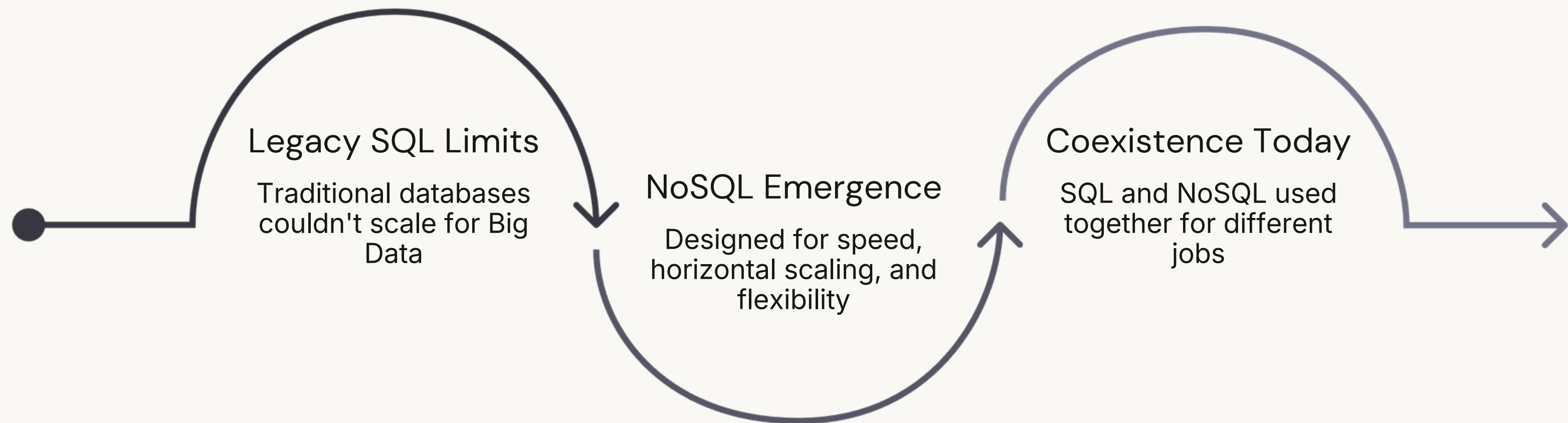
Netflix

Delivers **instant personalized** recommendations to 200M+ subscribers worldwide



Conclusion

NoSQL changed how the world handles data — not by replacing SQL, but by filling in the gaps SQL couldn't reach.



📌 **The big idea:** There's no single "best" database. The best engineers know when to use SQL, when to use NoSQL, and why each tool exists.